Feedforward Networks and Backpropagation

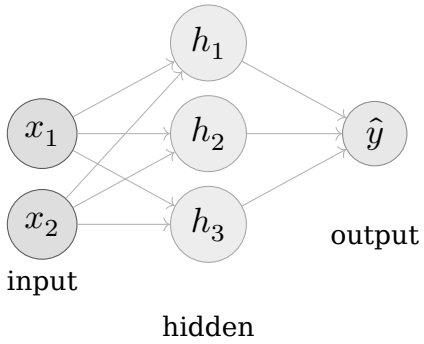
From the Chain Rule to ChatGPT: The Mathematics Behind AI

John Mitchell · Clark College · WAMATYC 2026

May 2026

Stacking Neurons into Layers

A single neuron can only learn a linear decision boundary — it cannot, for example, learn the XOR function. The solution is to stack neurons into layers, where the outputs of one layer become the inputs of the next. The result is a *feedforward network*, also called a multilayer perceptron.



Each hidden node is a neuron: it computes a weighted sum of its inputs, adds a bias, and applies an activation function. The network above has two input features, one hidden layer of three neurons, and one output neuron — a total architecture written as $2 \to 3 \to 1$.

Historical note. The theoretical foundations of multilayer networks were established by Rumelhart, Hinton, and Williams in their landmark 1986 paper introducing backpropagation to the machine learning community. While the chain rule itself is centuries old and backpropagation had been discovered independently several times (Werbos, 1974), the 1986 paper demonstrated that hidden layers could learn useful internal representations — resolving the limitation identified by Minsky and Papert in 1969. This is widely regarded as the start of the modern neural network era.

Matrix Notation for the Forward Pass

With many neurons per layer, writing out every weight individually becomes unwieldy. Matrix notation compresses the computation cleanly.

For a layer with n inputs and m neurons, collect the weights into a matrix $W \in \mathbb{R}^{m \times n}$ and the biases into a vector $\mathbf{b} \in \mathbb{R}^m$. The pre-activation output of the layer is:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

Applying the activation function elementwise gives the layer output:

$$\mathbf{h} = g(\mathbf{z})$$

For a two-layer network ($2 \to 3 \to 1$), the full forward pass is:

$$\mathbf{h} = g\big(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big) \qquad \hat{y} = \sigma\big(W^{(2)}\mathbf{h} + \mathbf{b}^{(2)}\big)$$

where $W^{(1)} \in \mathbb{R}^{3\times 2}$, $\mathbf{b}^{(1)} \in \mathbb{R}^3$, $W^{(2)} \in \mathbb{R}^{1\times 3}$, $\mathbf{b}^{(2)} \in \mathbb{R}^1$.

Key result. The forward pass is a composition of functions:

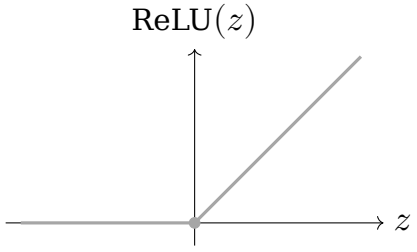
$$\hat{y} = f^{(2)}\big(f^{(1)}(\mathbf{x})\big)$$

Each $f^{(\ell)}$ is an affine transformation followed by an elementwise nonlinearity. The full network is a nested composition, which means the chain rule applies directly to compute its gradients.

ReLU: The Standard Hidden-Layer Activation

For hidden layers, the sigmoid activation has been largely replaced by the Rectified Linear Unit:

$$\text{ReLU}(z) = \max(0, z)$$



ReLU is preferred in hidden layers for two practical reasons. First, its gradient is exactly 1 for positive inputs and 0 elsewhere, which keeps gradients from shrinking as they propagate back through many layers — a problem called *vanishing gradients* that plagued sigmoid-based deep networks. Second, it is computationally cheap: a single comparison to zero.

> **Common misconception.** ReLU is not differentiable at $z = 0$. In practice this is not a problem — the probability of landing exactly at zero during training is negligible, and a subgradient of 0 is used at that point by convention. The more important property of ReLU is that its gradient does not saturate for large positive inputs, unlike sigmoid.

Parameter Count

The number of parameters in a feedforward network grows with depth and width. For a fully connected layer with n inputs and m neurons: mn weights plus m biases $= m(n+1)$ parameters.

For our $2 \to 3 \to 1$ network:

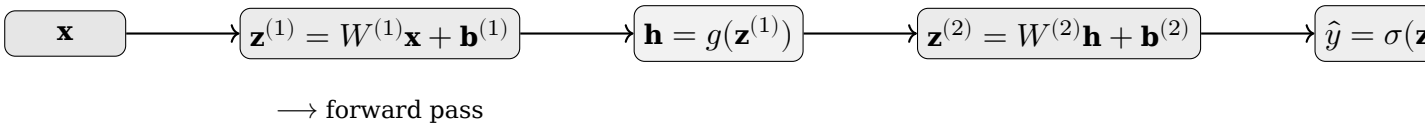
Layer	Computation	Parameters
Hidden ($2 \to 3$)	3×2 weights $+3$ biases	9
Output ($3 \to 1$)	1×3 weights $+1$ bias	4
Total		**13**

Thirteen parameters. By the end of this handout series, we will account for 175 billion. The same update rule governs all of them.

The Computation Graph

Before deriving the gradient, it helps to represent the forward pass as a *computation graph*: a directed acyclic graph where each node is an operation and edges carry values forward.

For the two-layer network, the forward pass decomposes as:



The computation graph makes the chain rule application visible: the loss L is a composition of functions, and the gradient of L with respect to any parameter is a product of local derivatives along the path from that parameter to the loss.

Backpropagation

Backpropagation is the chain rule applied systematically to a computation graph. It computes $\frac{\partial L}{\partial \theta}$ for every parameter θ in one efficient backward pass.

The Chain Rule, Revisited

For a composition $L = f(g(\theta))$, the chain rule gives:

$$\frac{dL}{d\theta} = \frac{dL}{df} \cdot \frac{df}{dg} \cdot \frac{dg}{d\theta}$$

In a computation graph, each edge carries one factor of this product. The gradient at any node is the product of all gradients along the path from that node to the loss — computed right to left.

One Backward Path

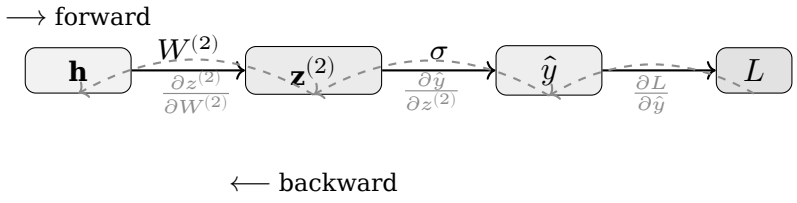
Consider the gradient of L with respect to the second-layer weights $W^{(2)}$. The path through the graph is:

$$L \leftarrow \hat{y} \leftarrow \mathbf{z}^{(2)} \leftarrow W^{(2)}$$

Applying the chain rule along this path:

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial W^{(2)}}$$

Each factor is a local derivative at one node — computable from quantities already known from the forward pass.



The same process continues leftward through the graph to reach $W^{(1)}$ — the gradient accumulates as a product of local terms, one per layer.

Going deeper: the full gradient for $W^{(1)}$. The path from $W^{(1)}$ to L passes

through every subsequent node. By the chain rule:

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial W^{(1)}}$$

Each factor is a matrix or vector of partial derivatives. The product $\frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial \mathbf{z}^{(1)}}$ carries gradient signal back through the hidden layer — and this is precisely where vanishing gradients occur if the activation function has small derivatives (as sigmoid does for large inputs).

Why One Pass Is Enough

A naive approach would compute $\frac{\partial L}{\partial \theta}$ separately for each parameter — requiring one forward pass per parameter. For 175 billion parameters, this is hopeless.

Backpropagation avoids this by reusing intermediate results. Each node computes its local gradient once and passes it leftward. The total cost of one backward pass is the same order as one forward pass. This is the computational miracle that makes deep learning tractable.

Key result. Backpropagation is the chain rule applied to a computation graph. It computes the gradient of the loss with respect to every parameter in the network in a single backward pass, with cost proportional to the forward pass. The update rule remains:

$$\theta \leftarrow \theta - \alpha \nabla_\theta L$$

The chain rule is what makes this update computable at any scale. This is why the chain rule matters more now than ever.

Historical note. The chain rule dates to Leibniz and Newton in the 17th century. Its application to layered networks was described by Werbos (1974) in his PhD thesis and independently by others, but it was the Rumelhart–Hinton–Williams (1986) paper that demonstrated the algorithm learning useful internal representations in practice. The word “backpropagation” refers to the direction of gradient flow: forward through the network during prediction, backward through the network during learning. Every major deep learning framework — PyTorch, TensorFlow, JAX — is built around automatic differentiation, which is a generalization of backpropagation to arbitrary computation graphs.

Summary: The Parameter Scaling Story So Far

Model	Parameters	Key idea
Linear regression	2	Loss + gradient descent
Single neuron	$d + 1$	Activation function
$2 \to 3 \to 1$ network	13	Composition of layers
$n \to m \to \cdots \to 1$	$O(nm)$ per layer	Backprop via chain rule
GPT-3	175 billion	Same update rule

The architecture grows in depth and width. The learning algorithm does not change. Handout 3 introduces the mechanism that allows transformers to handle sequential data — the attention mechanism — and with it the final large leap in parameter count.